

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4531 - Ağ Tabanlı Teknolojiler ve Uygulamaları
Gerçek Zamanlı Mesajlaşma Uygulaması

Açıklama Videosu

<https://www.youtube.com/watch?v=hhPfwm61Ec4>

Github Bağlantısı

<https://github.com/TimurMalkc/BLM4531-Chat-App>

Timur Malkoç – 22290719

İÇİNDEKİLER

1. GİRİŞ VE MOTİVASYON	3
2. KULLANILAN TEKNOLOJİLER VE KOD YAPISI	4
2.1. Kullanılan Teknolojiler.....	4
2.2. Kod Yapısının Yüzeysel Anlatımı	4
3. VERİ TABANI	6
4. API YAPILARI.....	8
5. PROJE ÖZELLİKLERİ	10
5.1 Gerçek Zamanlı İletişim	10
5.2 Kullanıcı Yönetimi ve Güvenlik	11
5.3 Bireysel ve Grup Halinde Mesajlaşma	13
5.4 Grup Yönetimi	15
5.5 Mesaj Geçmişi.....	20
5.6 Kullanıcı ve Oda Bilgileri.....	21
6. İKİNCİ GÖSTERİMDEN SONRA TAMAMLANAN EKSİKLER	22
6.1. Delete API Eklenmesi ve Swagger Üzerinde Gösterimleri	22
6.2. Tasarım Güncellemesi.....	22

1. GİRİŞ VE MOTİVASYON

Bu projenin amacı, SignalR kütüphanesinden faydalanarak Whatsapp Web veya Skype benzeri, web tabanlı ve gerçek zamanlı bir mesajlaşma uygulaması geliştirmektir. Proje, modern web teknolojileri ve ölçeklenebilir bir mimari yaklaşımı benimseyerek kullanıcıların bireysel veya grup tabanlı olarak mesajlaşabilmelerini sağlar.

Bu proje bugüne kadar geliştirdiğim en kapsamlı full stack uygulama oldu. Web socket ve gerçek zamanlı uygulama konseptlerini daha iyi öğrenebilmek için seçmiş olduğum bu proje sayesinde bu tarz uygulamalarda bulunan birçok mekanizmayı ve ortaya çıkan sorunların çözümlerini öğrendim.

2025-2026 öğretim yılının güz döneminin neredeyse tamamını kapsayan çalışmalarımı başlangıçta hayal ettiğim gibi popüler mesajlaşma uygulamalarına benzeyen yapıda bir proje elde ettim.

2. KULLANILAN TEKNOLOJİLER VE KOD YAPISI

Bu başlık altında; projede kullanılan teknolojilerden, projenin kod dosyalarının amaçlarından ve içeriklerinden yüzeysel olarak bahsedilmektedir. Detaylı kod açıklamaları anlatım videosunda mevcut olmaktadır.

2.1. Kullanılan Teknolojiler

Projede kullanılan ana teknolojiler aşağıdaki listede görülmektedir:

Backend ASP.NET Core Web API : ASP.NET Core Web API (Backend) Uygulamanın sunucu tarafını oluşturur ve kullanıcı, grup ve mesaj yönetimi gibi tüm HTTP tabanlı işlemleri gerçekleştirir. Frontend ve veritabanı arasındaki ana kontrol katmanıdır.

Gerçek Zamanlı İletişim SignalR : Kullanıcılar arasında gerçek zamanlı ve çift yönlü iletişim sağlar. Mesajların, grup güncellemelerinin ve kullanıcı durumlarının sayfa yenilemeye gerek kalmadan anında iletilmesini mümkün kılar.

Frontend HTML, CSS, JavaScript : Uygulamanın kullanıcı arayüzünü ve etkileşimli yapısını oluşturur. SignalR ve API istekleri JavaScript üzerinden yönetilerek gerçek zamanlı sohbet deneyimi sağlanır.

Veri Tabanı MongoDB : Kullanıcılar, gruplar ve mesajlar gibi verileri doküman tabanlı yapıda saklar. Esnek veri modeli sayesinde sohbet uygulamasına uygun ve hızlı veri erişimi sunar.

Kimlik Doğrulama JWT : Kullanıcıların güvenli bir şekilde kimlik doğrulamasını sağlar. Oluşturulan token sayesinde yetkili API ve SignalR işlemleri güvenli biçimde gerçekleştirilir.

2.2. Kod Yapısının Yüzeysel Anlatımı

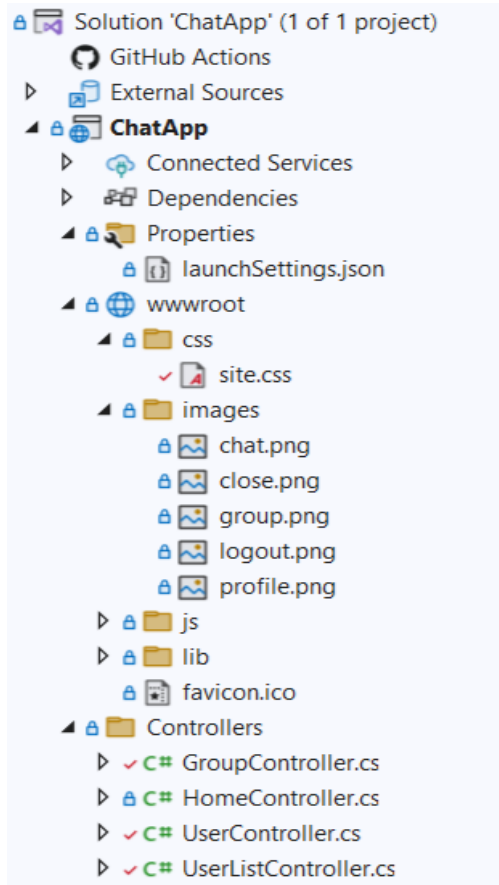
Controller Sınıfları: HTTP tabanlı API uç noktalarını içerir ve kullanıcı, grup ve listeleme gibi işlemleri yönetirler. İstemciden gelen istekleri alarak gerekli servis katmanına yönlendirirler.

SignalR Hub (ChatHub): Gerçek zamanlı mesajlaşma ve bağlantı yönetimini sağlar. Kullanıcıların odalara katılması, ayrılması ve anlık mesajların iletilmesi bu yapı üzerinden gerçekleştirilir.

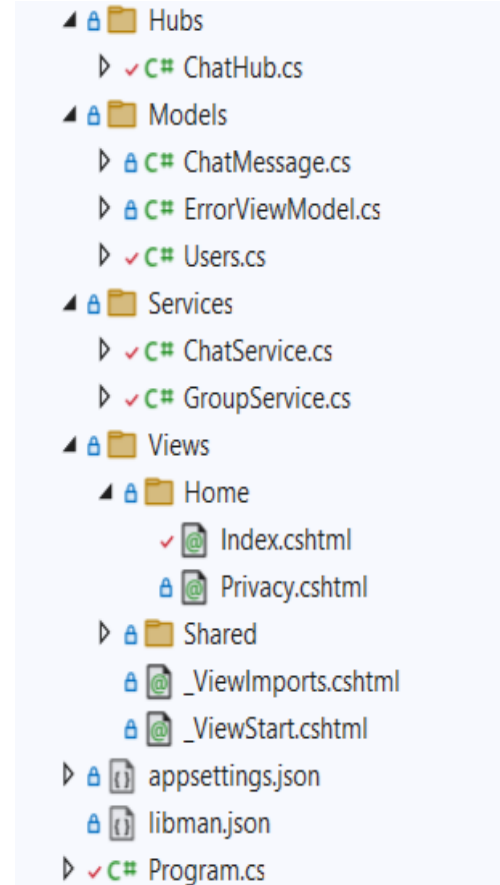
Model Sınıfları: MongoDB'de tutulan verilerin C# tarafındaki temsilini oluştururlar. Kullanıcı, mesaj ve grup gibi veri yapıları bu sınıflar aracılığıyla tanımlanır.

Service Sınıfları: Controller ve Hub katmanları ile veritabanı arasındaki iş mantığını yönetir. MongoDB erişimi ve veri üzerinde yapılan tüm işlemler bu katman üzerinden gerçekleştirilir.

Index.cshtml Sınıfı: Uygulamanın ana kullanıcı arayüzünü ve frontend mantığını içerir. SignalR bağlantıları, API çağrıları ve sohbet arayüzünün dinamik davranışları bu dosyada yönetilir.



Şekil 1.1. Kod yapısının birinci parçası



Şekil 1.2. Kod yapısının ikinci parçası

3. VERİ TABANI

Projede MongoDB veri tabanı kullanılmıştır. MongoDB, doküman tabanlı ve esnek yapısı sayesinde mesaj, kullanıcı ve grup gibi verilerin hızlı ve şemasız şekilde yönetilmesini sağlar. Gerçek zamanlı mesajlaşma uygulamalarında sık okuma yazma işlemleri için projenin başlangıç aşamasında kullanılması düşünülen SQL Server'a göre daha esnek ve ölçeklenebilir bir çözüm sunar.

MongoDB'de Bulunan Temel koleksiyonlar:

Groups:

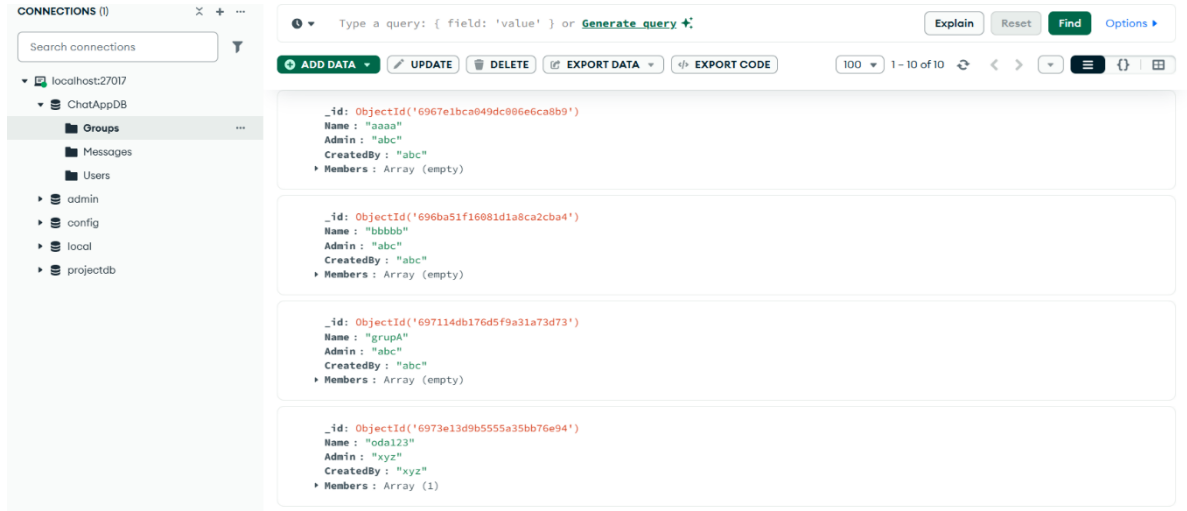
- Grup adı
- Grup kurucusu (birden fazla yönetici eklenmesi halinde üst bir yetki olsun diye farklı bir kategori olarak açılmıştır, böyle bir sistemin eklenilmemesine karar verildiği için şu an özel bir işlevi yoktur)
- Grup yöneticisi
- Grup üyeleri

Messages:

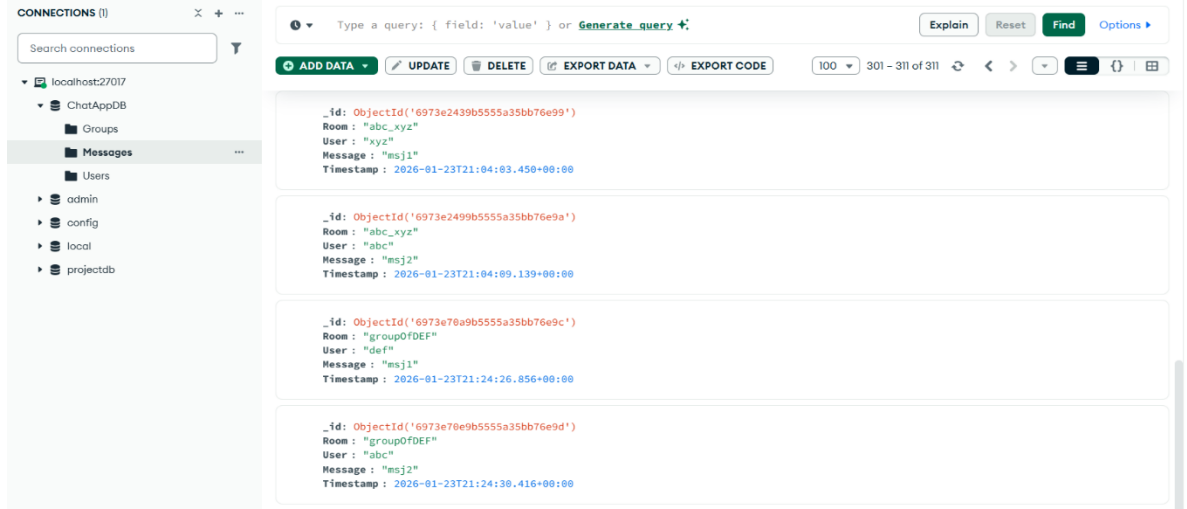
- Gönderen kullanıcı bilgisi
- Alıcı kullanıcı veya grup bilgisi
- Mesaj içeriği
- Gönderim tarihi ve saati

Users:

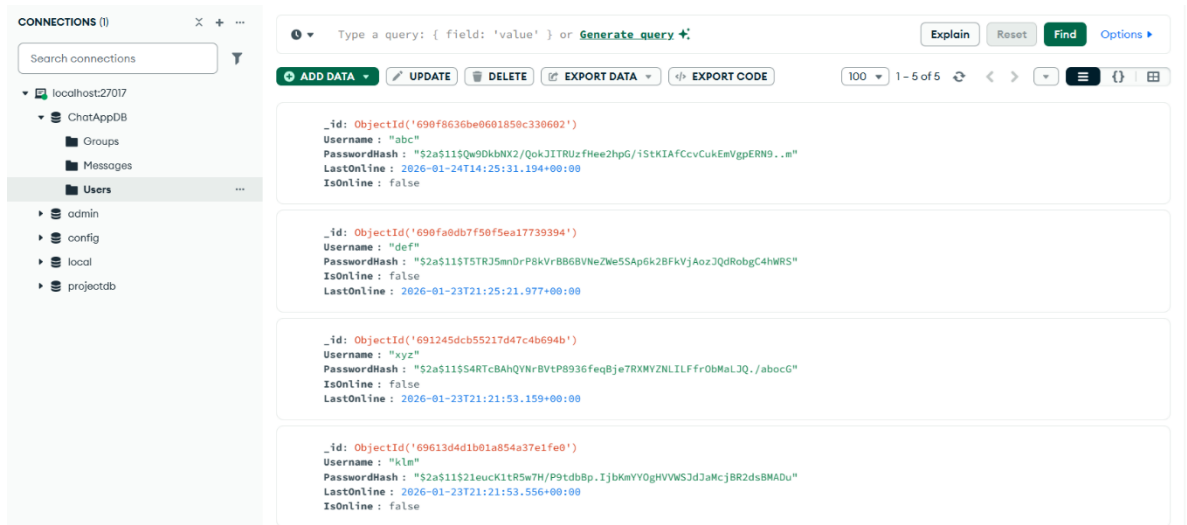
- Kullanıcı adı
- Gizlenmiş şifre
- Son giriş tarihi
- Çevrimiçi olma durumu
- Kullanıcının dahil olduğu gruplar



Şekil 2.1. Groups koleksiyonu



Şekil 2.2. Messages koleksiyonu



Şekil 2.3. Users koleksiyonu

4. API YAPILARI

Group API Grupları: GroupController sınıfı içerisinde bulunan bu API'lar sırasıyla belli bir kullanıcı için bütün grupları sıralama, grup oluşturma, gruplara üye ekleme, gruplardan üye çıkarma, var olan bir grubu silme, ismi verilen grup objesini döndürme ve gruplardan ayrılma işlemlerini gerçekleştirirler.

User API Grupları: UserController sınıfı içerisinde bulunan bu API'lar sırasıyla yeni üye kaydı, kullanıcının sisteme giriş yapması, kullanıcı bilgilerinin çekilmesi, hesaptan çıkış yapılması ve hesap silinmesi işlemlerini gerçekleştirirler.

UserList API Grubu: UserListController sınıfı içerisinde bulunan tek API sisteme kayıtlı olan bütün kullanıcılara erişir ve döndürür.

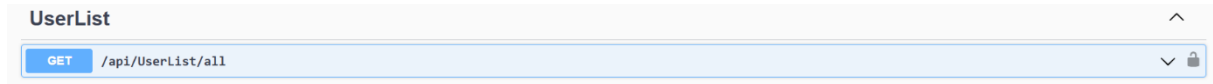
Schemas: DTO yapılarını kullanarak üye ekleme, grup oluşturma, gruptan çıkma, giriş ve kayıt yapma işlemleri için gerekli bilgilerin standart bir formatta taşınmasını sağlarlar.

Group		^
GET	/api/group	✓ 🔒
POST	/api/group/create	✓ 🔒
POST	/api/group/addUser	✓ 🔒
DELETE	/api/group/{groupName}/users/{username}	✓ 🔒
DELETE	/api/group/delete/{groupName}	✓ 🔒
GET	/api/group/{groupName}	✓ 🔒
POST	/api/group/leave	✓ 🔒

Şekil 3.1. Group API grupları

User		^
POST	/api/User/register	✓ 🔒
POST	/api/User/login	✓ 🔒
GET	/api/User/info/{username}	✓ 🔒
POST	/api/User/disconnect	✓ 🔒
DELETE	/api/User/delete	✓ 🔒

Şekil 3.2. User API grupları



Şekil 3.3. UserList API grubu

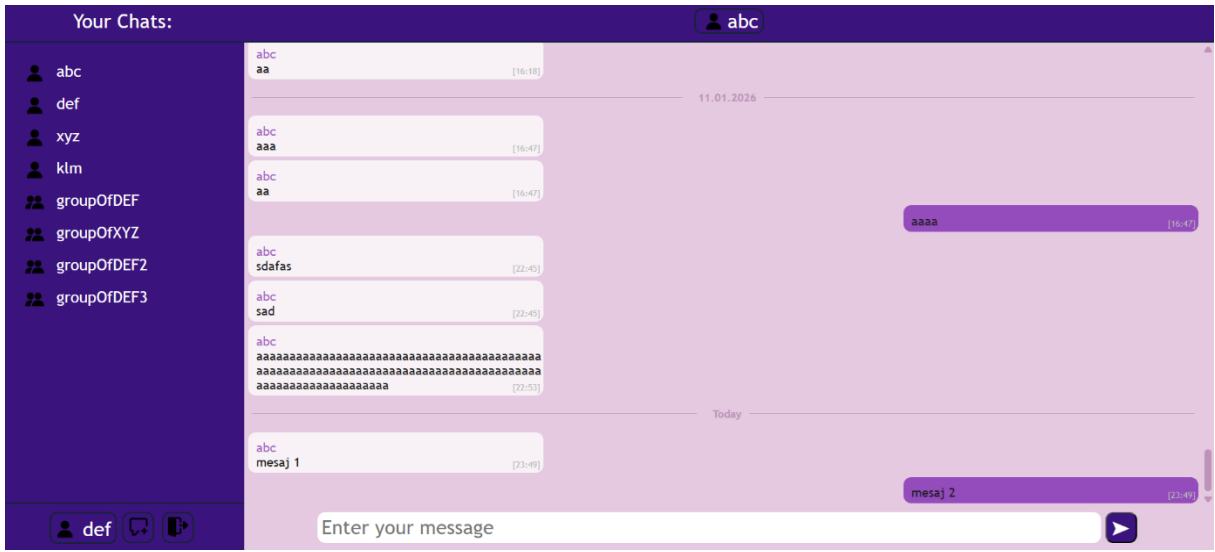


Şekil 3.4. Schemas

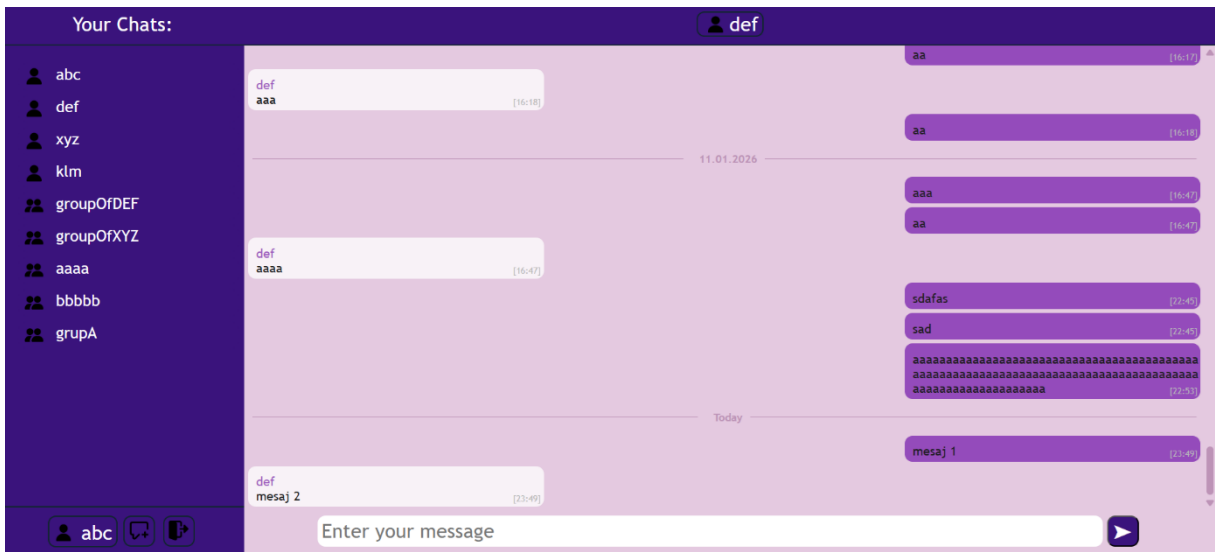
5. PROJE ÖZELLİKLERİ

5.1. Gerçek Zamanlı İletişim

SignalR altyapısı sayesinde mesajlar sayfa yenilemeye gerek olmadan anlık olarak iletilmektedir. Bireysel odalarda sadece iki kullanıcı arasında, grup odalarında ise bağlı olan herkes arasında kurulan bağlantılarla kişi sınırı olmadan mesajlar herkese aynı anda iletilir. İletişim için çevrimiçi olma zorunluluğu yoktur ve gönderilen mesajlar veri tabanına kaydedildiğinden kullanıcılar çevrimdışı oldukları anlarda gönderilen mesajlara da erişebilirler.



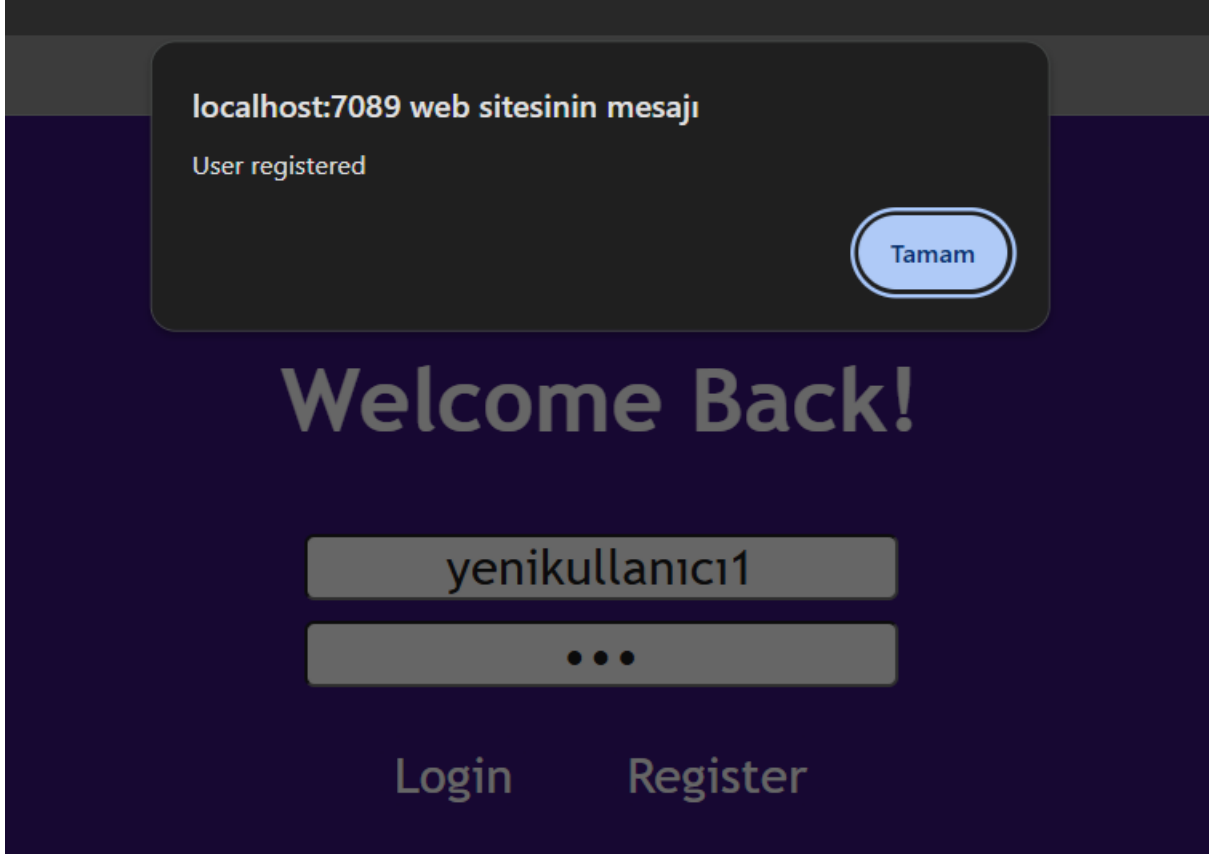
Şekil 4.1.1. Sohbetin “def” kullanıcısının bakış açısından görünümü



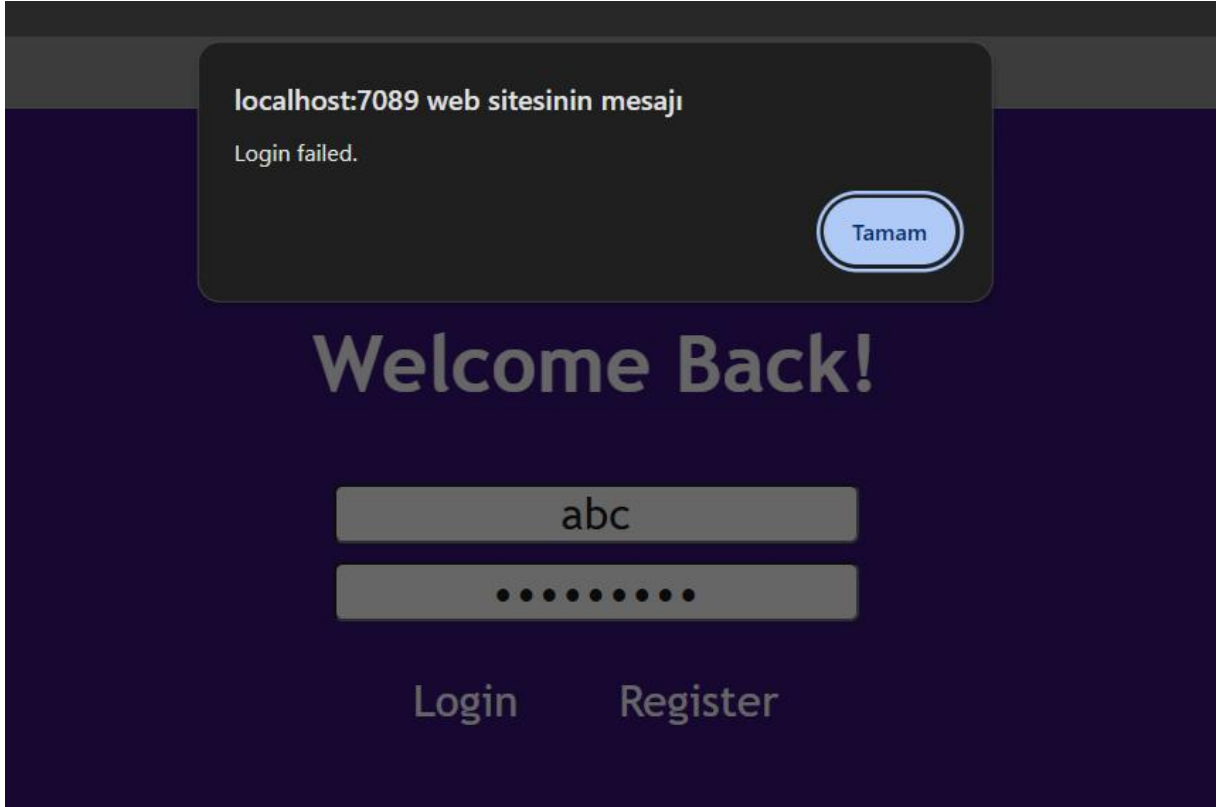
Şekil 4.1.2. Sohbetin “abc” kullanıcısının bakış açısından görünümü

5.2. Kullanıcı Yönetimi ve Güvenlik

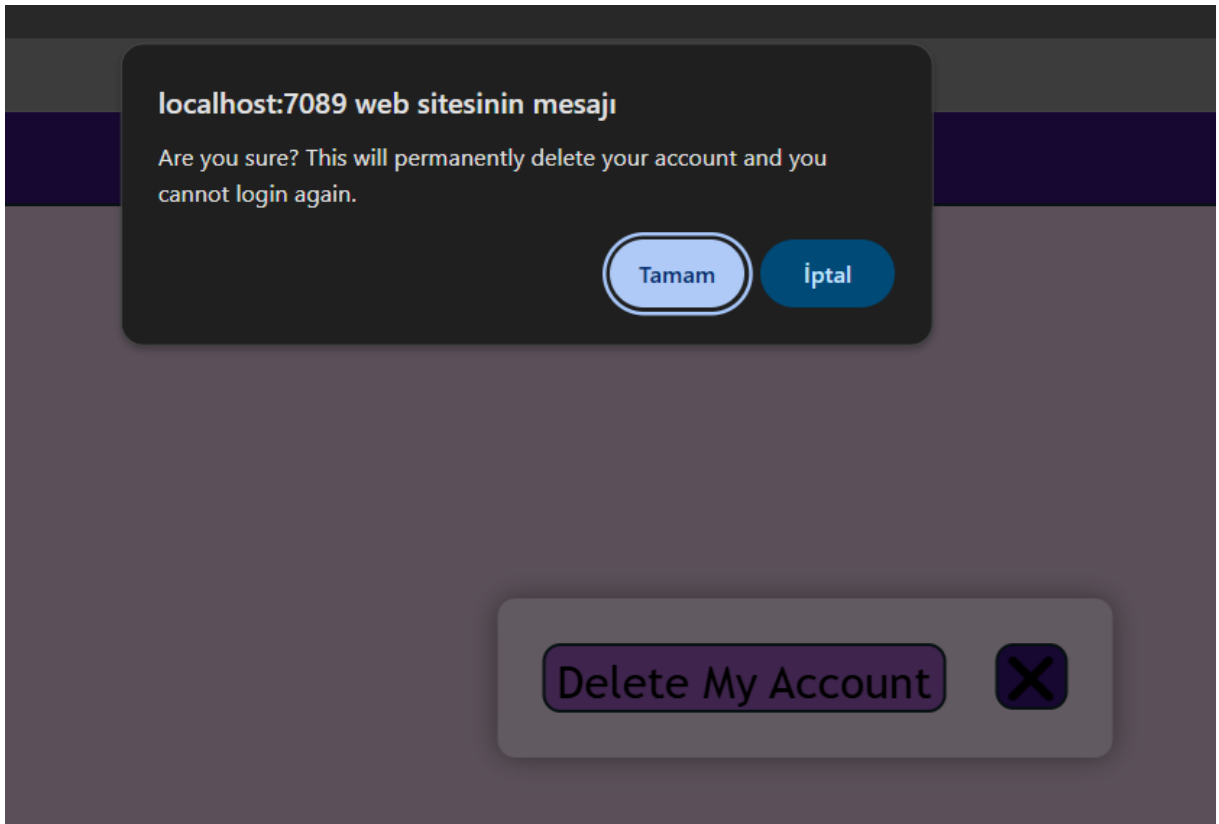
Kullanıcılar sisteme yeni hesaplar ile kaydolabilir, mevcut hesaplarıyla giriş yapabilir ve hesaplarını silebilirler. Kullanıcıların şifreleri gizliliği korumak amacıyla SHA384 hash algoritması ile şifrelenerek saklanır. Kullanıcı kimlik doğrulama ve yetkilendirme işlemleri JWT (JSON Web Token) kullanılarak sağlanmıştır.



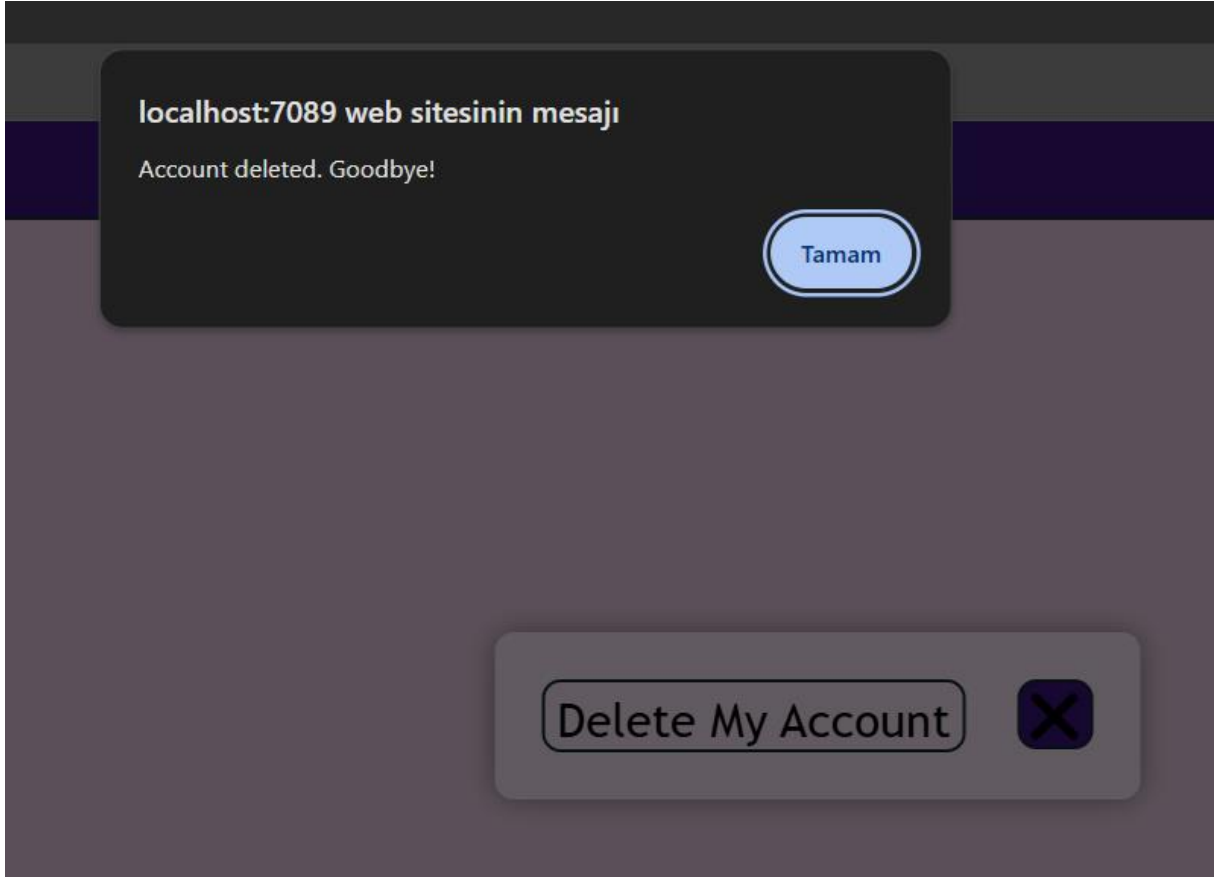
Şekil 4.2.1. Başarılı kaydolma durumu



Şekil 4.2.2. Yanlış şifre yüzünden başarısız giriş denemesi durumu



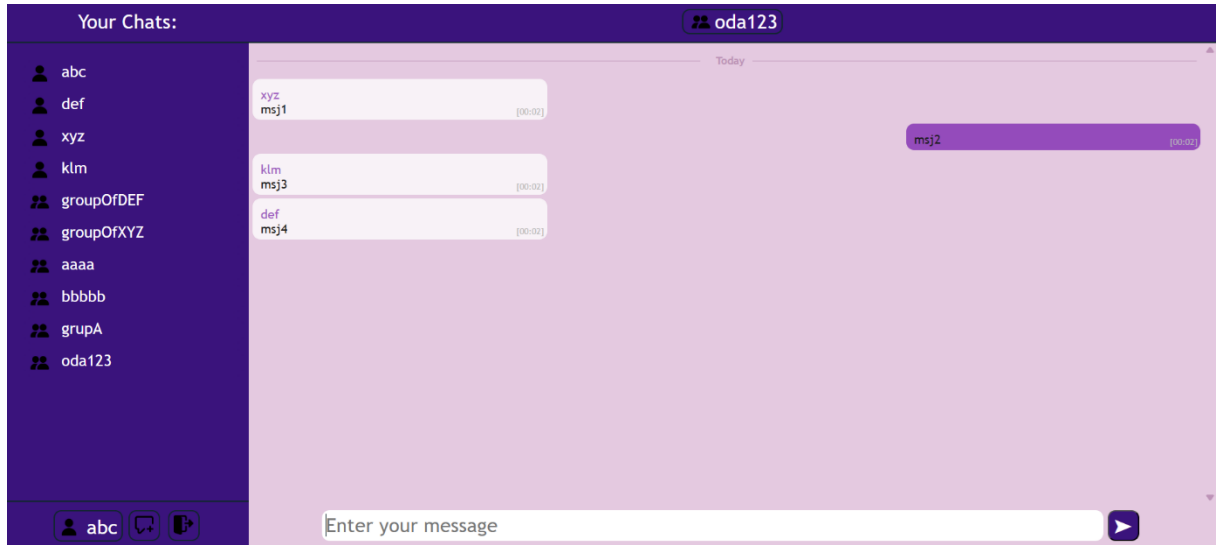
Şekil 4.2.3. Hesap silme işlemini başlatma durumu



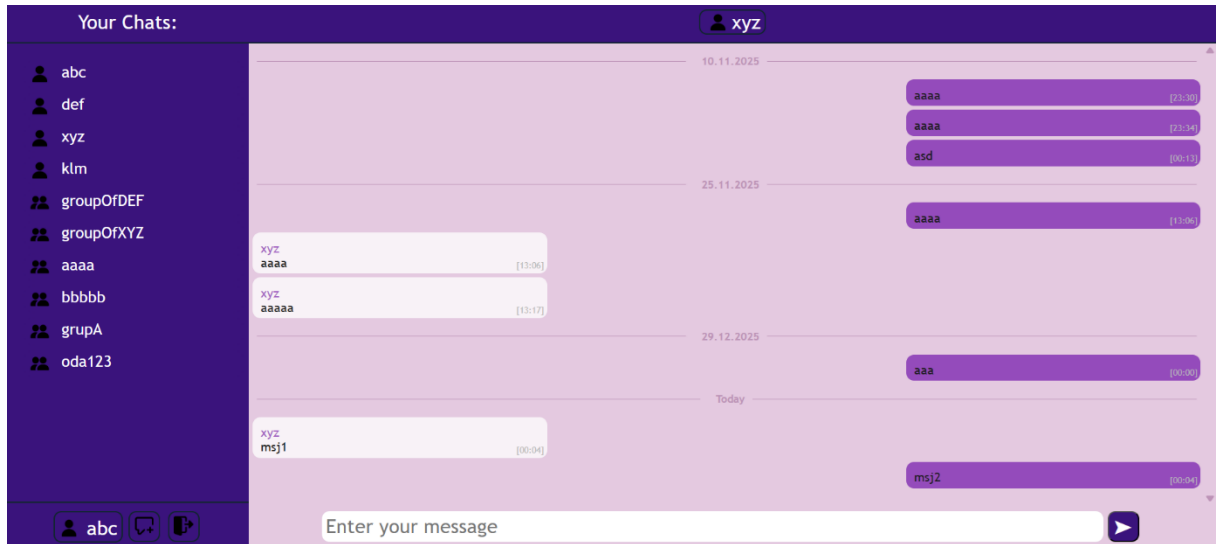
Şekil 4.2.4. Başarılı bir şekilde hesap silme durumu

5.3. Bireysel ve Grup Halinde Mesajlaşma

Kullanıcılar bireysel sohbet odalarında veya grup sohbetlerinde mesajlaşabilmektedir. Kullanıcıların sistemde bulunan ve kendileri de dahil olmak üzere bütün kullanıcılara erişimi vardır. Kullanıcı, alıcının çevrimiçi olup olmama durumundan tamamen bağımsız olarak her an birine veya bir gruba mesaj gönderebilir. Kullanıcılar yalnızca dahil oldukları grupları görüntüleyebilir ve sadece bu gruplarda mesajlaşabilirler.



Şekil 4.3.1. Grup sohbet odası görünümü



Şekil 4.3.2. Bireysel sohbet odası görünümü

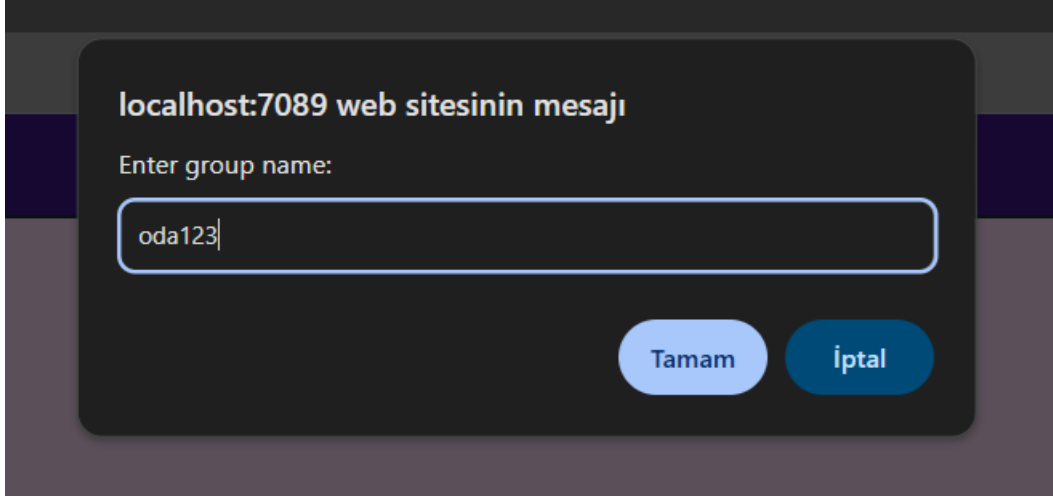


Şekil 4.3.3. Oda listelerinin farklı kullanıcılar için farklı görünüşleri

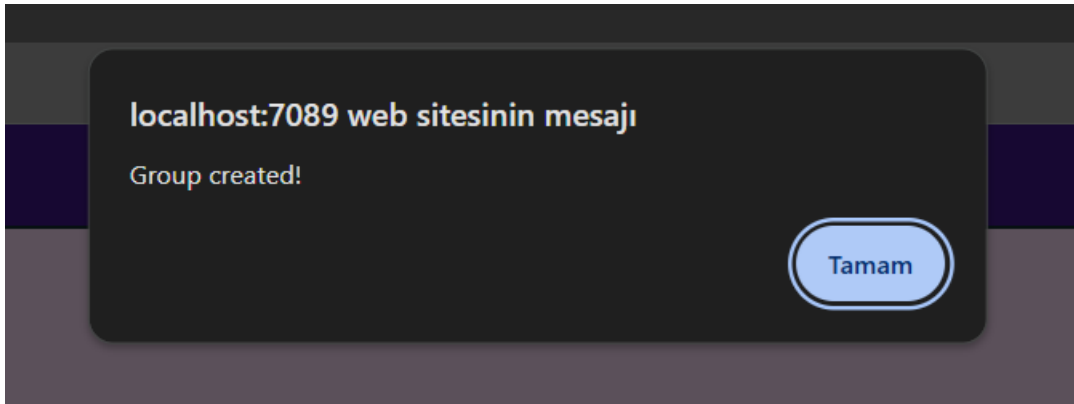
5.4. Grup Yönetimi

Her kullanıcının yeni bir grup oluşturma yetkisi bulunmaktadır. Grup kurucuları yeni üyeler ekleyebilir, mevcut üyeleri gruptan çıkarabilir veya grupları tamamen silebilirler. Kullanıcılar kendi istekleri ile gruplardan ayrılabilir. Uygulamada kurucu yetkisi devretme sistemi eklenmediği için kurucu üyeler oluşturdukları gruplardan sadece

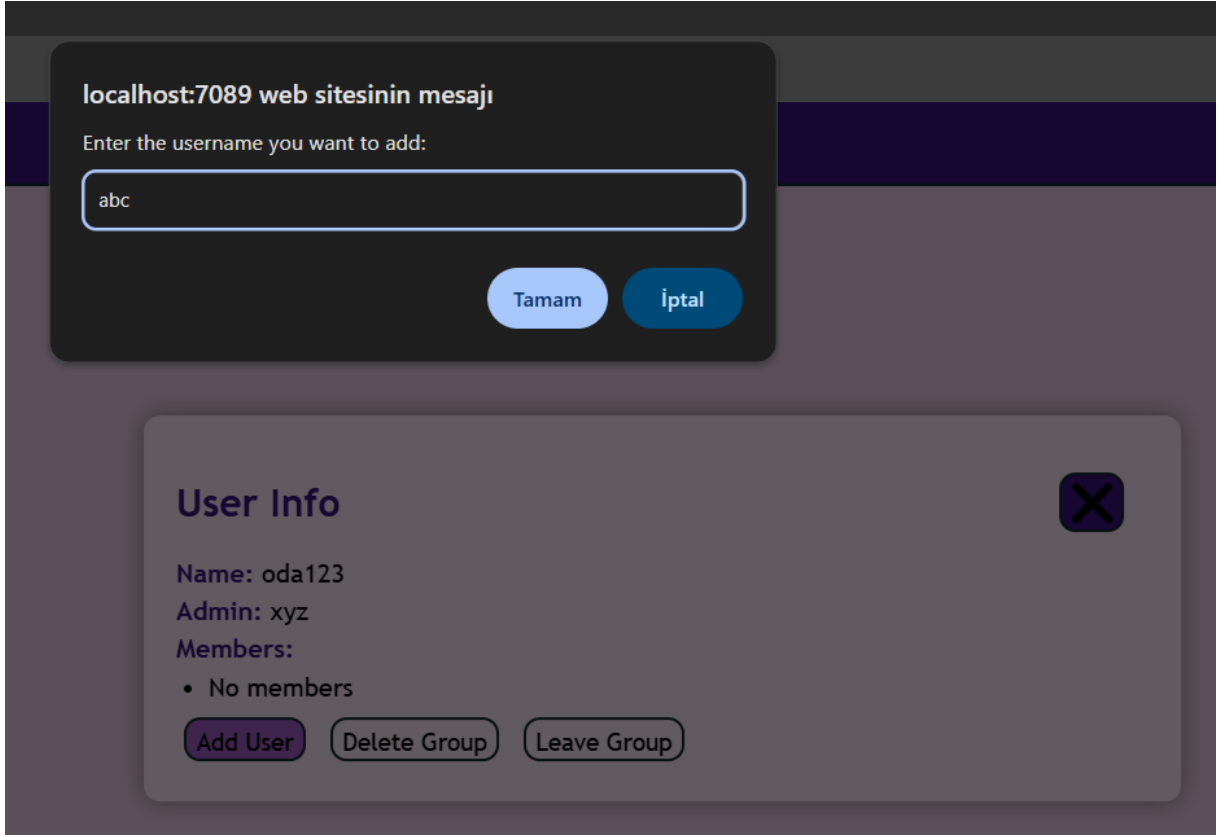
grubu silerek çıkabilirler. Gruptan kendi istekleri çıkan veya kurucu tarafından çıkarılan üyeler artık bu odaların eski mesajlarını göremez, yeni mesajlar gönderemez, gönderilen yeni mesajlara erişemez ve bu odayı geri açamazlar. Çıkarılan kullanıcının odayı geri açamamasına rağmen mesajlara dair böyle kısıtlar getirilmesinin sebebi, kullanıcının odadan çıkarıldığı an sohbetin ekranda açık olması ihtimalidir.



Şekil 4.4.1. Grup oluşturma işleminin başlangıcı



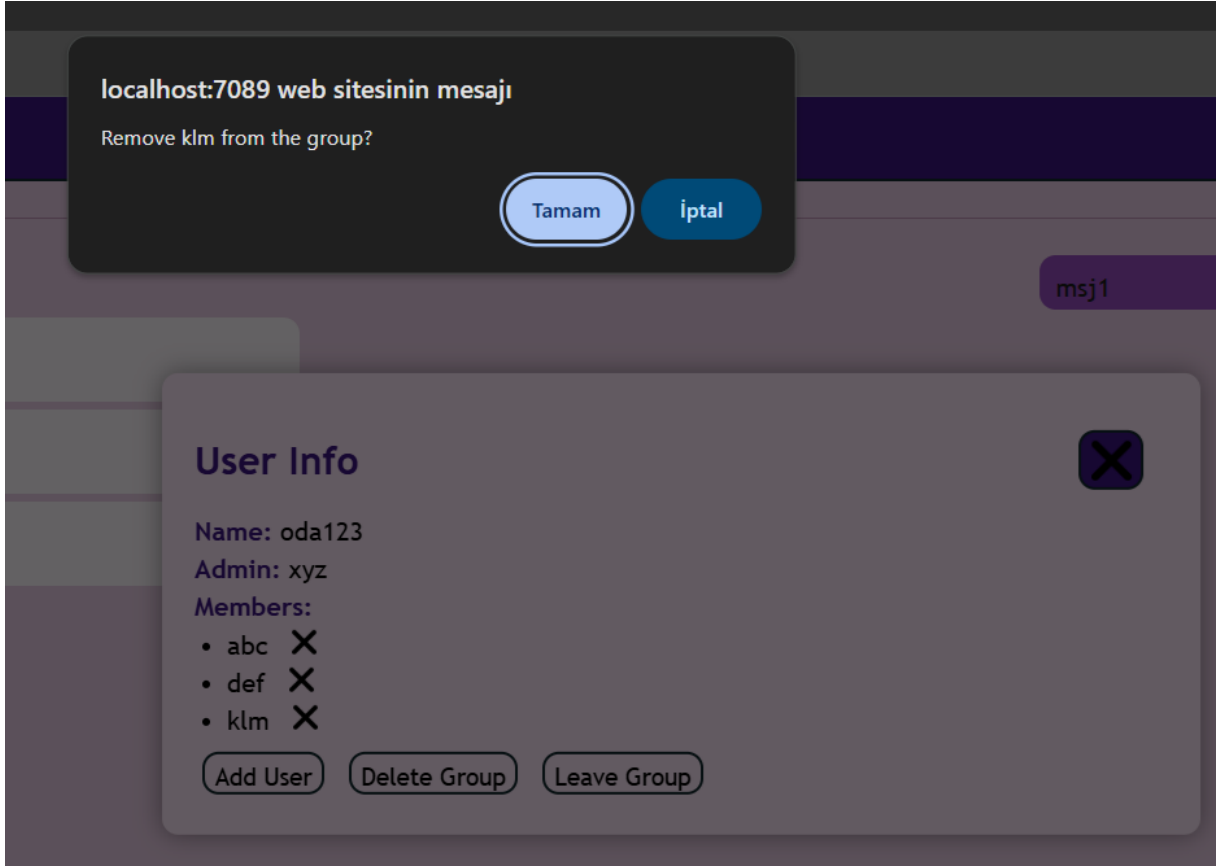
Şekil 4.4.2. Başarılı grup açma durumu



Şekil 4.4.3. Üye ekleme işleminin başlangıcı



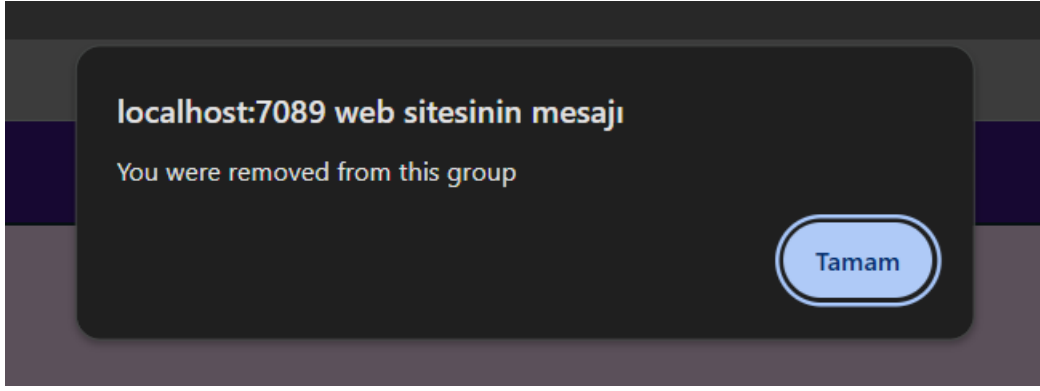
Şekil 4.4.4. Üyenin başarılı eklendiği durum



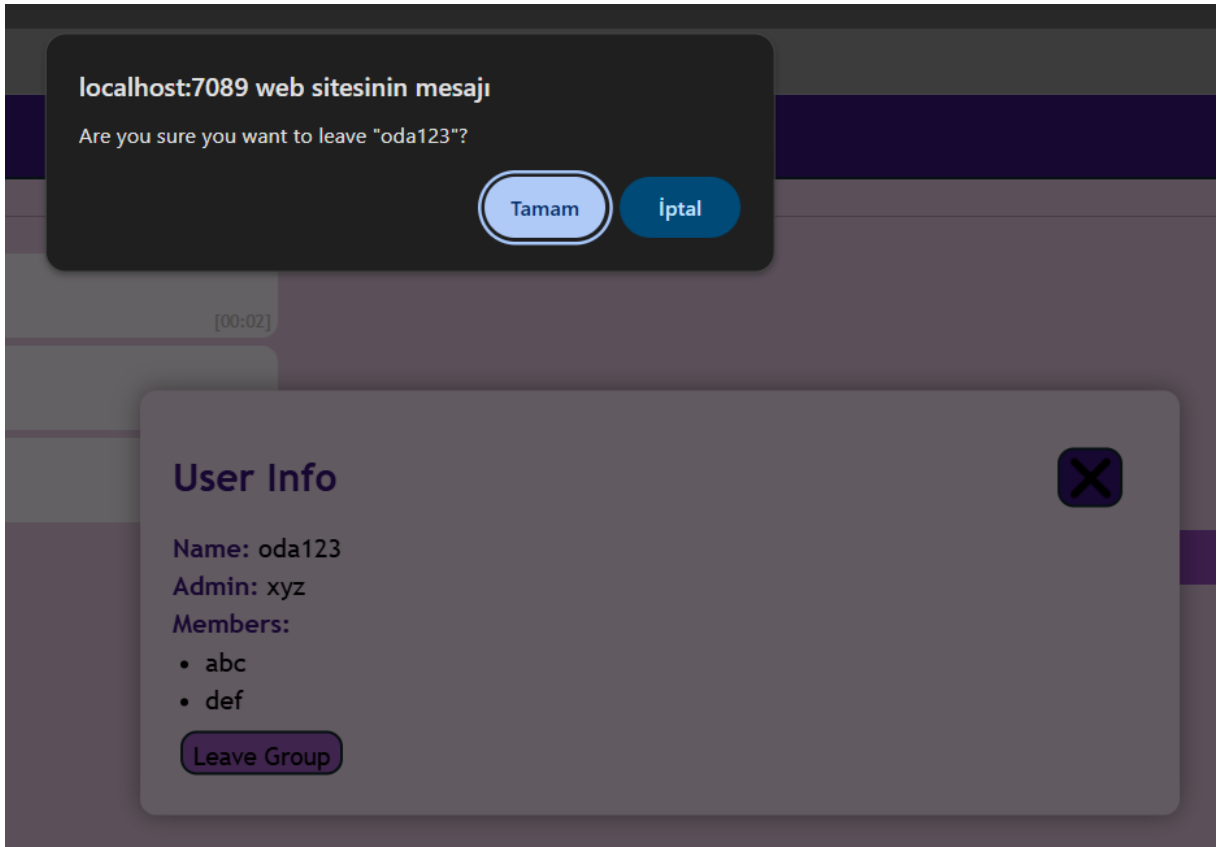
Şekil 4.4.5. Üyenin gruptan atılma işleminin başlangıcı



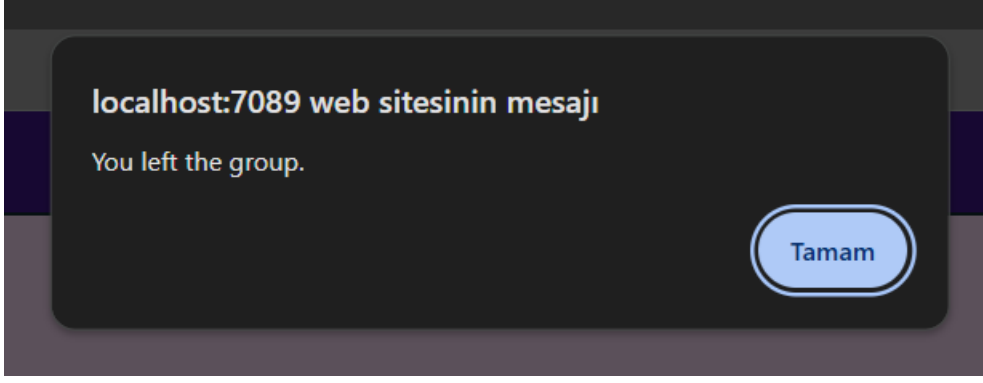
Şekil 4.4.6. Üyenin gruptan atılma işleminin başarılı durumu



Şekil 4.4.7. Odadan çıkarılan üyeye gönderilen mesaj



Şekil 4.4.8. Kullanıcının kendi isteğiyle odadan çıkması durumunun başlangıcı



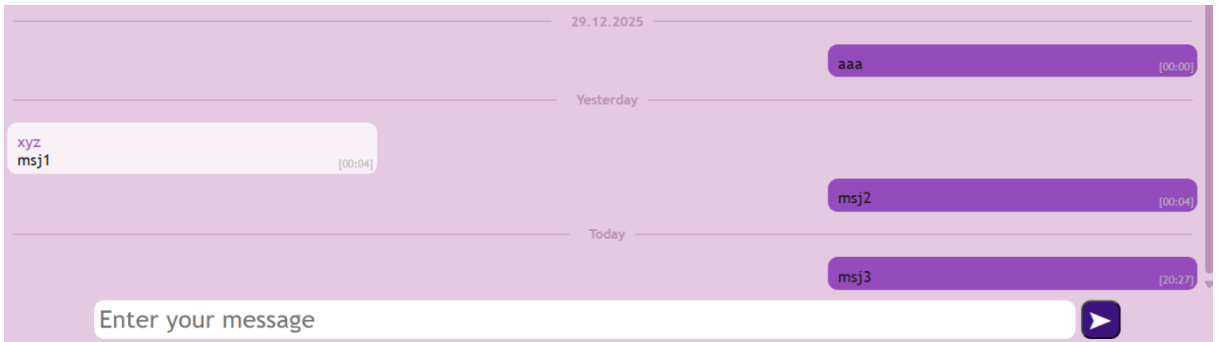
Şekil 4.4.9. Kullanıcının odadan çıkmasının başarılı gerçekleşmesi durumu



Şekil 4.4.10. Gruptan çıkma işleminden sonra erişilemeyen odanın görüntüsü

5.5. Mesaj Geçmişi

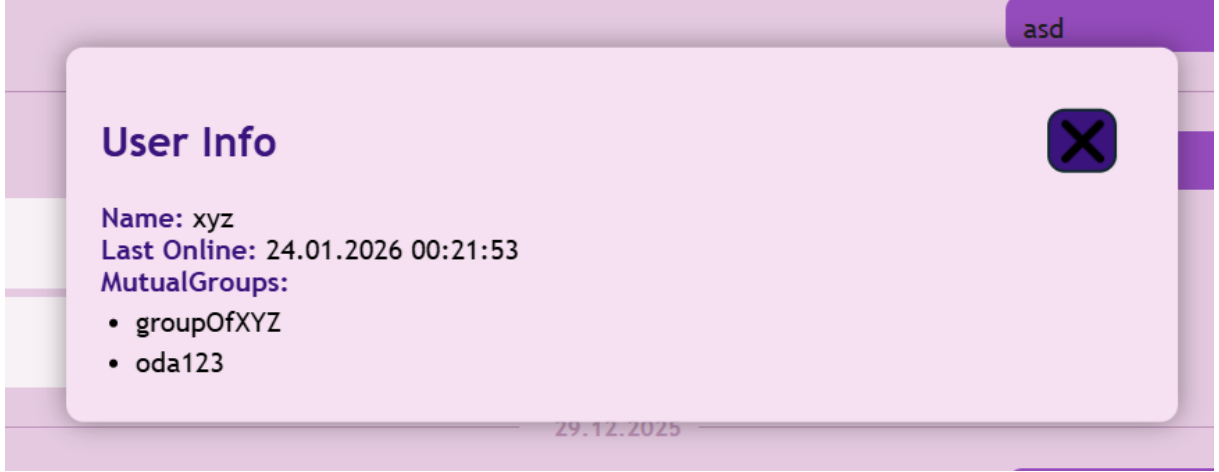
MongoDB veri tabanı sayesinde eski mesajlar gönderim saatleri ile yüklenmektedir. Mesajların gün farklılıkları kullanıcı arayüzünde görsel olarak ayırt edilecek şekilde gösterilmektedir.



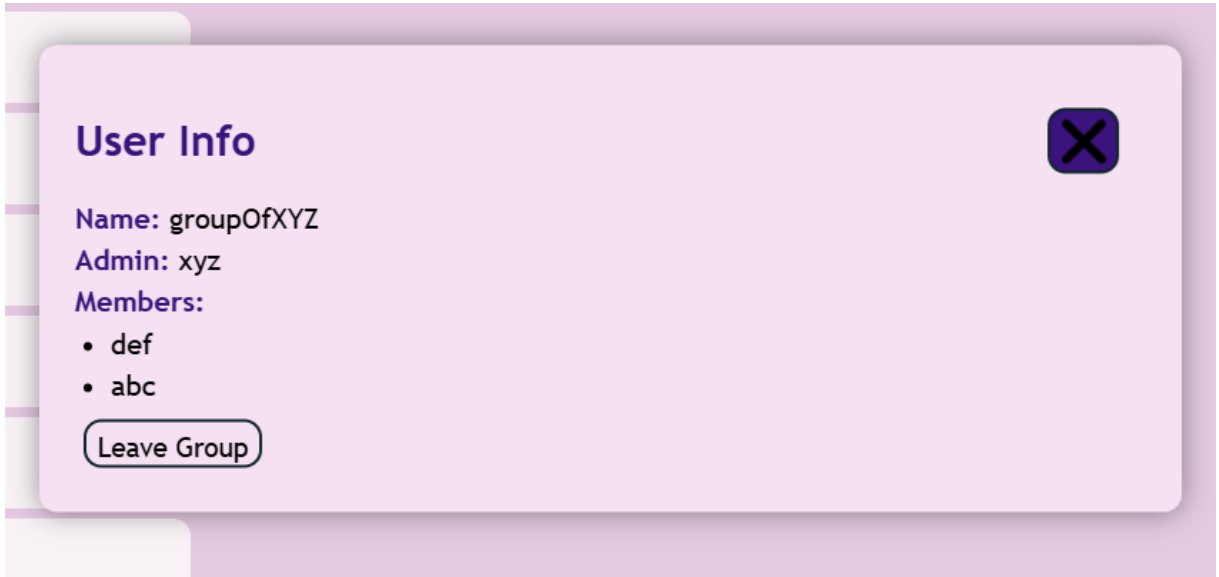
Şekil 4.5.1. Mesajların gün ayrımı

5.6. Kullanıcı ve Oda Bilgileri

Kullanıcılar; birbirlerinin isim, ortak grup bilgileri ve son giriş tarihi gibi bilgilerine erişebilmektedir. Kullanıcılar grup sohbetleri için aynı düğme üzerinden grupların adı, admini ve üyeler listesi gibi bilgilere erişebilirler.



Şekil 4.6.1. Kullanıcı bilgilerine erişim



Şekil 4.6.2. Grup odası bilgilerine erişim

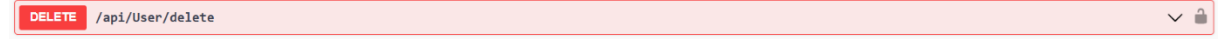
6. İKİNCİ GÖSTERİMDEN SONRA TAMAMLANAN EKSİKLER

6.1. Delete API Eklenmesi ve Swagger Üzerinde Gösterimleri

İkinci gösterimin ardından projeye; gruptan kullanıcı silme, grup sohbet odası silme ve kullanıcı hesabı silme olmak üzere üç tane DELETE API yapısı eklenmiştir.



Şekil 5.1.1. Gruptan kullanıcı silme ve grup sohbet odası silme API yapıları



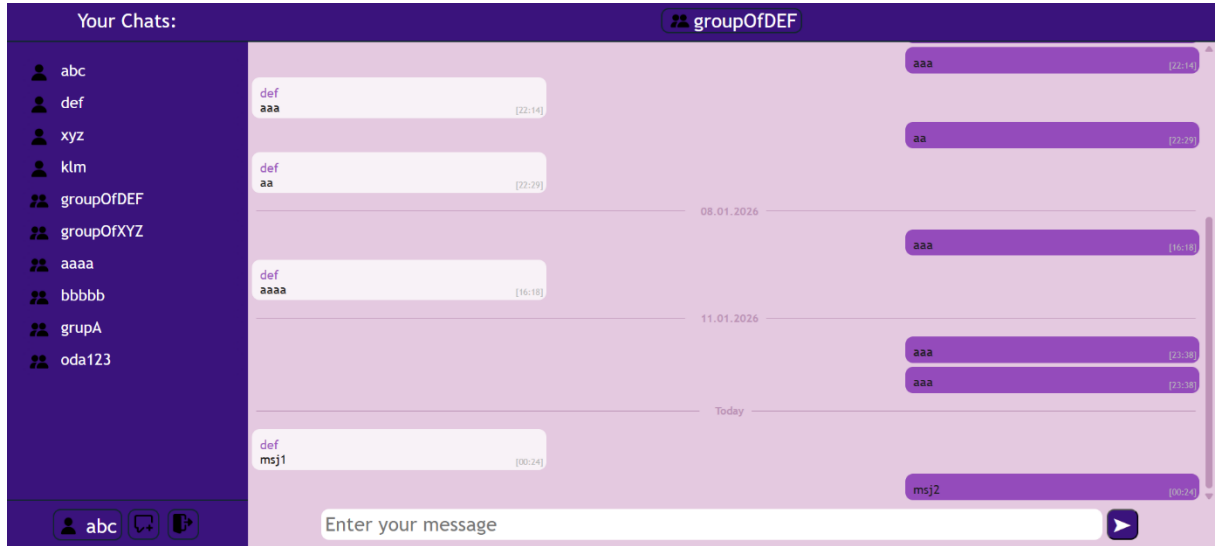
Şekil 5.1.2. Kullanıcı hesabı silme API yapısı

6.2. Tasarım Güncellemesi

İkinci gösterimin ardından birçok tasarım değişikliği yapılmıştır. Daha modern ve rahat bir görünüm elde etmek için birbiriyle daha uyumlu ve çakışmayan renkler seçilmiş, fazlalık oluşturan renk grupları azaltılmıştır. Mesaj balonlarının, liste öğelerinin ve çerçeve alanlarının boyutları küçültülmüştür. Yazı boyutları da aynı şekilde küçültülmüş ve gerekli yerlerde renkleri değiştirilmiştir. Gün farklılıklarının anlaşılması için gün ayraçları eklenmiştir. Tasarımı anlaşılır kılmak için eklenen rastgele emoji ler birbirleriyle uyumlu olacak şekilde ikon görselleri ile değiştirilmiştir.



Şekil 5.2.1. İkinci gösterimde kullanılan tasarım



Şekil 5.2.2. Final tasarımı